APLICAÇÃO INTERATIVA EM TEMPO REAL

Parque da Reciclagem 3D

Da geometria ao movimento:
os conceitos por trás de um jogo educativo.

Fernando Buligon · José C. Seben



PYTHON + VPYTHON

EXPLORE. RECOLHA. SEPARE.

O que o trabalho pede — e onde aparece

Para cada requisito: **conceito → implementação → exemplo no jogo.**

I · SLIDES 05–09

Geometria e espaço

Coordenadas e transformações; malhas; modelagem hierárquica.

II · SLIDES 10–11

Visualização e interação

Câmera, projeção perspectiva e navegação.

III · SLIDES 12–16

Estética e renderização

Iluminação, sombreamento, materiais e texturas.

IV · SLIDES 17–19

Dinâmica

Transformações e animações atualizadas pelo tempo.

> A defesa exige explicar a matemática que a ferramenta abstrai. O mapa completo está no apoio 23.

Uma partida: recolher, classificar e descartar

Limpar o parque levando **seis resíduos** às três lixeiras.

01 · PAPEL

Folha e caixa

Identificação azul + texto

02 · PLÁSTICO

Garrafa e copo

Identificação vermelha + texto

03 · VIDRO

Garrafa e pote

Identificação verde + texto

+100 pontos

Descarte correto: o item entra na lixeira.

−25 pontos

Lixeira errada: o item permanece com o jogador.

Um item por vez

A vitória acontece após seis acertos; sem erros, são 600 pontos.

Python calcula; o navegador mostra

Entrada

Teclado, arraste do mouse e roda.

→

Estado e pose

Python atualiza regras, colisões, câmera e animação.

→

Imagem 3D

VPython envia a cena ao renderizador WebGL no navegador.

`main.py`
Jogador, jogo, cenário e loop

`modelos_residuos.py`
Malhas por revolução

`colisoes.py · iluminacao.py`
Cálculos independentes da cena

`animacao.py`
Poses e cinemática inversa

`carregador_stl.py`
Leitura e conversão de modelos

`menu.* · mouse.js · janela.js`
Interface e eventos do navegador

Cada objeto ocupa um lugar no espaço

O referencial da cena

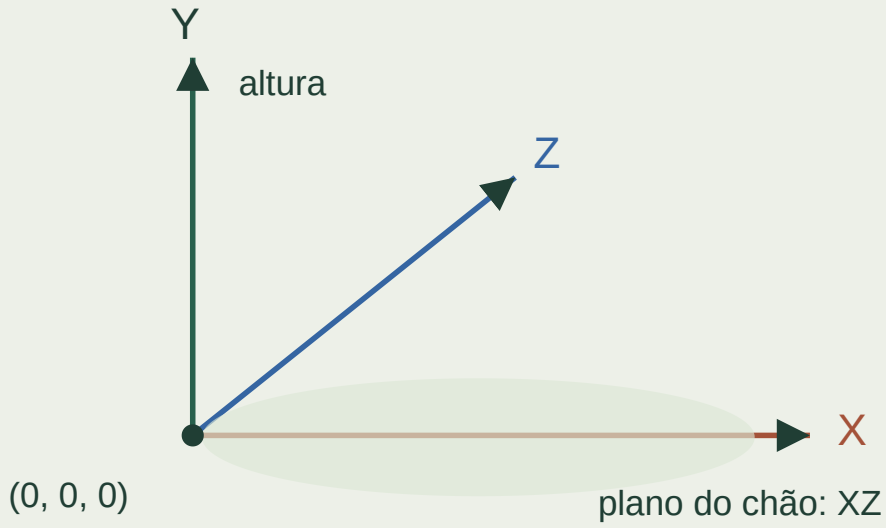
X e Z definem o chão; Y representa a altura. O parque mede 34 × 26 unidades.

Translação

Mover é somar um deslocamento à posição: `pos + direção × velocidade × Δt`.

Rotação e escala

O personagem gira em torno de Y. As peças giram nas articulações; os STL são ajustados pela altura.



> Coordenadas locais descrevem a peça; coordenadas de mundo dizem onde ela está no parque.

A ordem das transformações muda o resultado

$$p_{mundo} = T \cdot R \cdot S \cdot p_{local}$$

Convenção: vetores coluna e coordenadas homogêneas (x, y, z, 1). A aplicação ocorre da direita para a esquerda.

1. Escalar

Ajustar a forma e o tamanho no espaço local.

2. Rotacionar

Orientar o objeto em torno da origem local.

3. Transladar

Levar o objeto para sua posição no parque.

R e depois T

p = (1, 0, 0) → (0, 0, −1) → (2, 0, −1)

T e depois R

p = (1, 0, 0) → (3, 0, 0) → (0, 0, −3)

Exemplo: R = giro de +90° em Y; T = deslocamento de +2 em X. As matrizes completas estão no apoio 24.

Uma malha reúne forma e conectividade

Geometria: onde estão os pontos?

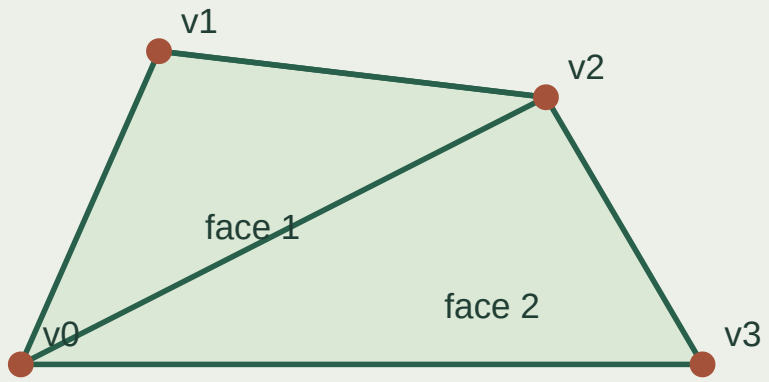
São as posições (x, y, z) dos vértices. Mover um ponto muda a forma.

Topologia: como os pontos se conectam?

Vértices, arestas e faces definem a estrutura. A mesma conectividade pode ter outra forma.

Três caminhos no projeto

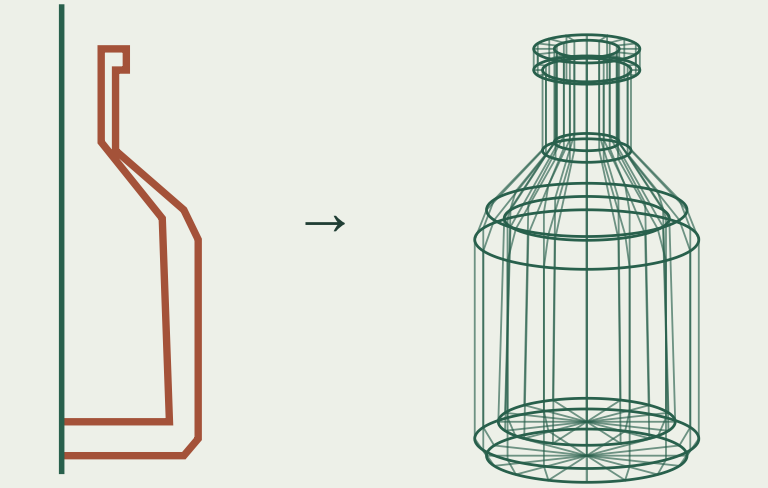
Primitivas do VPython, malhas geradas por código e 19 modelos STL externos.



Conectividade + posições = malha

> STL armazena triângulos; não traz materiais nem coordenadas UV. O importador calcula normais e aplica cores.

Garrafas e copos nascem de um perfil



perfil (r, y) → malha de revolução

Girar um contorno ao redor de Y

Para cada ponto (r, y), são calculadas posições em 16 ângulos ao redor do eixo.

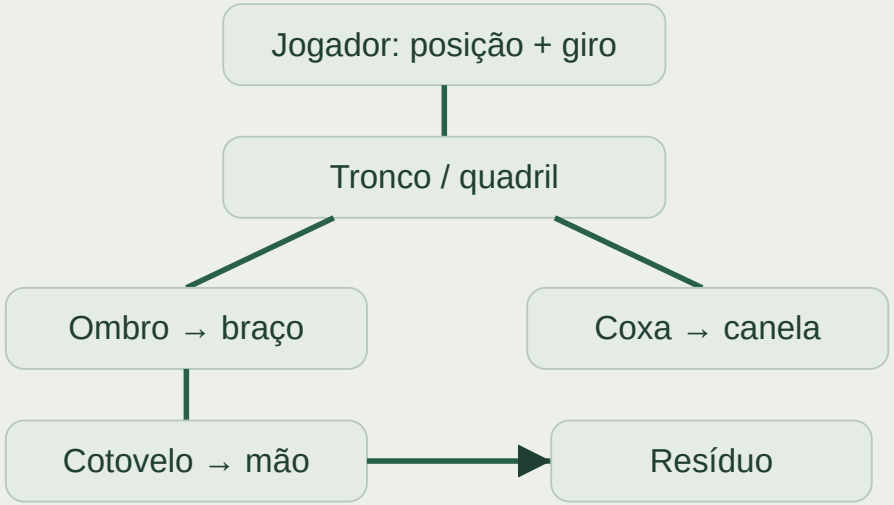
$$v(r, y, \theta) = (r \cos \theta, y, r \sin \theta)$$

Ligar os anéis

Cada faixa é triangulada; no eixo, faces degeneradas são evitadas. Paredes internas dão abertura real aos recipientes.

São quatro perfis de revolução; papel e caixa usam primitivas. Cada resíduo fica abaixo de 500 triângulos.

As peças dependem umas das outras



Transformações encadeadas

A orientação do tronco afeta o ombro; o braço define o cotovelo; o antebraço define a mão.

$$M_{filho,mundo} = M_{pai,mundo} \cdot M_{filho,local}$$

Como o código faz

Calcula os pivôs e ângulos em ordem, depois atualiza `pos`, `axis` e `up` de cada peça.

> Há uma hierarquia calculada manualmente. Não existe um grafo de cena genérico com herança automática.

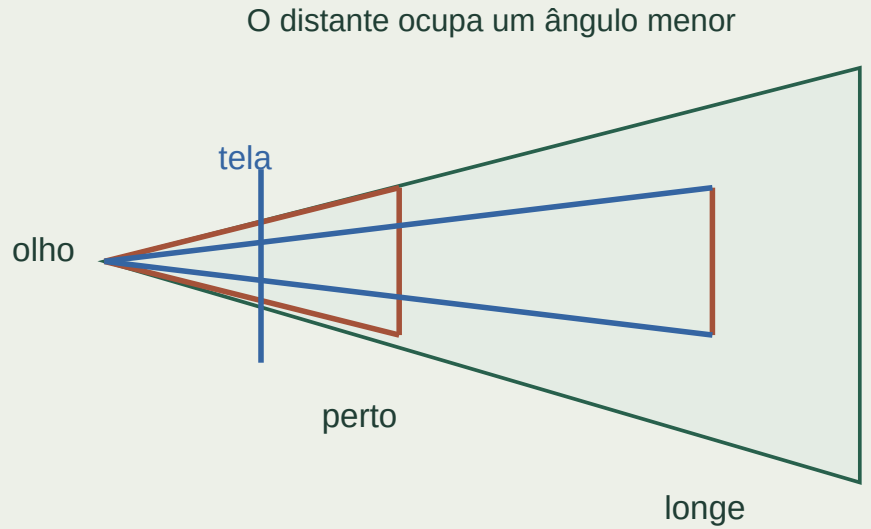
Da cena 3D à imagem em perspectiva

Câmera em terceira pessoa

A posição fica atrás do jogador; `camera.axis = alvo − câmera` aponta para a região à frente.

$$p_{clip} = P \cdot V \cdot M \cdot p_{local}$$

$$(x, y, z)_{NDC} = (x_c, y_c, z_c) / w_c$$



Por que o distante parece menor?

Na perspectiva, w acompanha a profundidade. A divisão por w reduz o tamanho projetado ao se afastar.

O jogo configura posição, direção e FOV (58°–74°); a matriz de projeção e a divisão por w ficam no renderizador.

Navegar também exige limites e interação

W **S** frente / ré **A** **D** giro **E** coletar / descartar **M** ou **Esc** pausa **R** reinício

Mouse

Arrastar gira o personagem; a câmera acompanha. A roda altera a distância entre 5 e 14 unidades.

Proximidade

Coleta até 1,35 unidade; descarte até 1,8. A seleção compara distâncias ao quadrado.

Colisões aproximadas

Círculos e retângulos bloqueiam obstáculos no plano XZ. Passos são subdivididos e permitem deslizar pelas bordas.

$$\text{frente} = (\sin\theta, 0, \cos\theta) \quad \cdot \quad \Delta p = \text{frente} \cdot v \cdot \Delta t$$

Manter E pressionado produz uma interação por toque, evitando penalidades repetidas.

A luz revela a orientação das superfícies

Quatro luzes direcionais + ambiente compõem uma iluminação diurna fixa.

Sol

Define a direção principal e a leitura do volume.

Preenchimento e céu

Suavizam o lado oposto e iluminam superfícies voltadas para cima.

Reflexo da grama

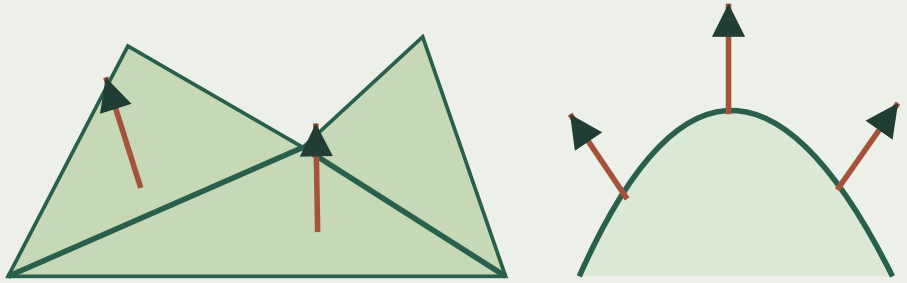
Uma luz direcionada de baixo simula a contribuição do chão.

$$I = k_a I_a + \Sigma\, I_l [k_d \max(0, N \cdot L) + k_s \max(0, R \cdot V)^n]$$

Modelo empírico de Phong: ambiente + difuso + especular; N, L, R e V são vetores unitários.

> O produto N·L mede quanto a superfície está voltada para a luz. O VPython executa o cálculo de iluminação.

Normais e sombreamento: duas escolhas distintas



normais por face

normais suaves

$$N_{face} = \text{normalizar}((b - a) \times (c - a))$$

Forma aparente

STL: mesma normal nos três vértices de cada face, preservando o estilo facetado. Resíduos: normais variam ao redor dos anéis, suavizando a superfície.

Gouraud: por vértice

Calcula iluminação nos vértices e interpola as cores. Pode perder brilhos que ficam no interior do triângulo.

Phong shading: por fragmento

Interpola normais, normaliza e calcula a luz em cada fragmento. É o caminho observado no shader padrão do VPython.

> Normais suaves não acrescentam triângulos. “Phong shading” descreve onde calcular; o modelo de Phong descreve a luz.

Materiais controlam a resposta à luz

SUPERFÍCIE	PARÂMETROS REAIS	EFEITO PROCURADO
Chão e caminhos	`shininess = 0`	Acabamento fosco
Modelos STL	`shininess = 0.15`	Brilho discreto e faces legíveis
Resíduos de vidro	`shininess = 0.55`	Brilho; recipientes opacos
Água do lago	`shininess = 0.8` · `opacity = 0.72`	Superfície brilhante e translúcida

Ambiente · k_a

Resposta à luz uniforme da cena.

Difuso · k_d

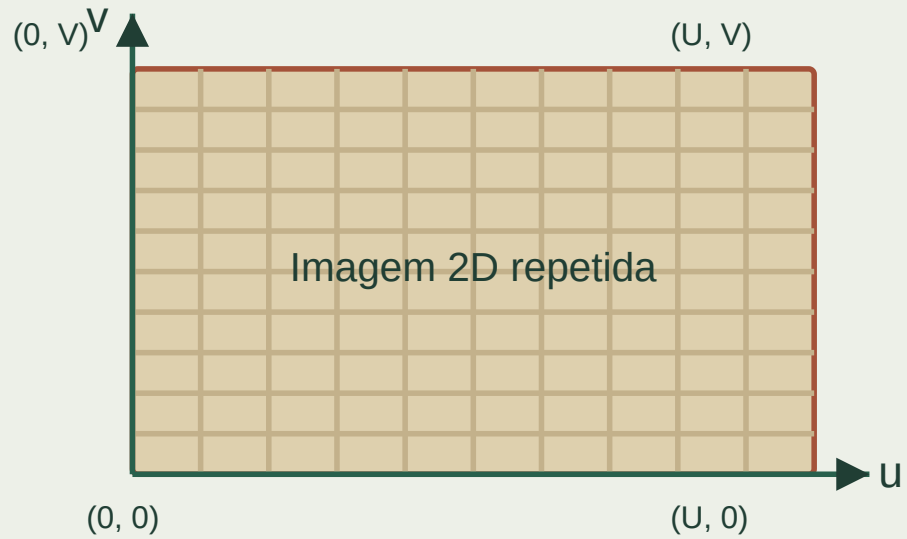
Cor da superfície e dependência de N·L.

Especular · k_s, n

Reflexo concentrado, dependente da direção do observador.

No código, o controle ocorre por color, shininess, opacity e scene.ambient. Não há k_a, k_d e k_s independentes por material.

As texturas já usam coordenadas UV



Um ponto na malha, um ponto na imagem

Cada vértice do caminho recebe `texpos = vector(u, v, 0)`; u acompanha X e v acompanha Z.

$$U = \text{largura} / 1{,}5 \quad \cdot \quad V = \text{profundidade} / 1{,}5$$

Texturas aplicadas

Cascalho nos caminhos; madeira nos bancos; pedras no chafariz. Bancos e chafariz usam o mapeamento padrão das primitivas.

> O UV repete o cascalho a cada 1,5 unidade. Aplicar cor a um objeto ou ao rótulo de uma garrafa não é aplicar uma textura.

Sombra no chão é diferente de sombreamento

Sombreamento

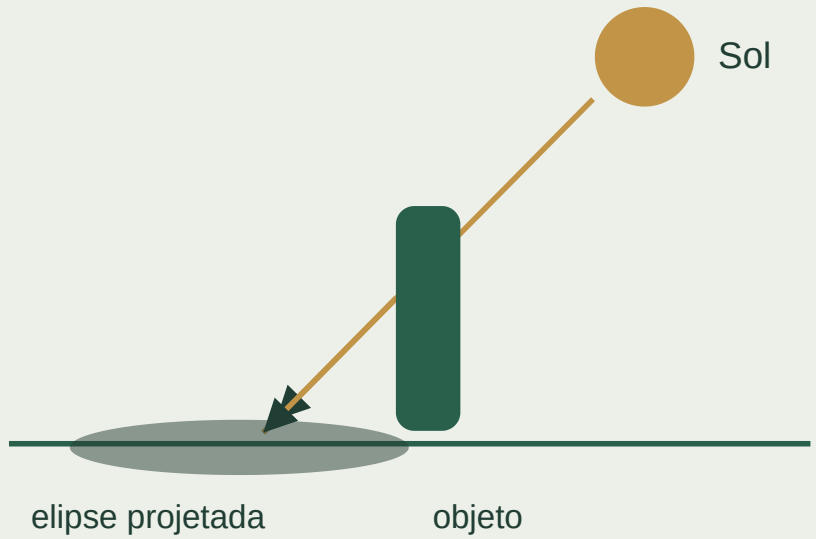
Variação de cor na própria superfície, calculada com luzes, materiais e normais.

Sombra do objeto

Uma elipse escura achatada representa a projeção no chão. O deslocamento cresce com a altura.

$$(\Delta x, \Delta z) = -h \cdot (L_x, L_z) / L_y$$

Projeção de um ponto a altura h no chão; a elipse aproxima a extensão do objeto.



> Não há shadow mapping, oclusão entre malhas ou contorno exato da sombra. É uma aproximação geométrica.

Movimento deve avançar pelo tempo

$$\Delta p = \text{direção} \cdot \text{velocidade} \cdot \Delta t$$

Medir o intervalo

`perf_counter()` mede o tempo desde a atualização anterior.

Avançar o estado

Posição, giro, nuvens e ações usam Δt. `rate(60)` apenas limita o loop.

Suavizar transições

`α = 1 − exp(−kΔt)` aproxima corpo, câmera e tampas do alvo.

30 atualizações/s	60 atualizações/s
4,5 × (1/30) × 30 = 4,5 unidades/s	4,5 × (1/60) × 60 = 4,5 unidades/s

Limite real: Δt é cortado em 0,05 s. Abaixo de 20 FPS sustentados, o jogo desacelera em relação ao tempo real.

O pé é o alvo; o joelho é calculado

Caminhada

A fase cresce com a distância realmente percorrida. Braços alternam em oposição às pernas.

Coleta

O quadril abaixa e o tronco inclina. Os tornozelos recebem alvos próximos ao chão.

Cinemática inversa

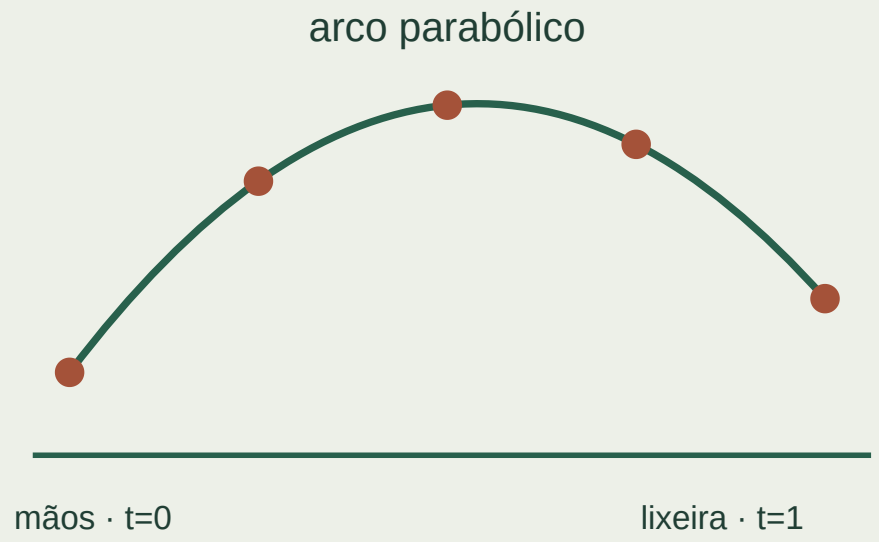
A lei dos cossenos encontra os ângulos da coxa e da canela para alcançar o alvo.

$$\cos \beta = (a^2 + d^2 - b^2) / (2ad)$$

a = comprimento da coxa; b = comprimento da canela; d = distância do quadril ao tornozelo; β = abertura entre a coxa e a direção do alvo.

> Alvos fora do alcance são limitados. A pose calculada é convertida em posição e orientação de cada peça.

Coletar e descartar são transições contínuas



Do chão às mãos

A coleta interpola a posição do item; o descarte só é liberado após sua fixação nas mãos.

Das mãos à lixeira

O voo dura 0,45 s. A posição combina uma linha entre origem e destino com uma elevação parabólica.

$$p(t) = (1-t)A + tB + (0,\ 4Ht(1-t),\ 0)$$

t vai de 0 a 1; H = 0,9 é a elevação adicional máxima. A pontuação muda no acerto; o modelo some ao terminar o voo.

Validação e escolhas para manter a cena leve

46

testes Python passaram

Nenhum teste ignorado na validação desta revisão.

Geometria e importação

19 STL reais, faces inválidas, orientação, fechamento e normais dos perfis.

Regras e animações

Coleta, erro/acerto, reinício, pausa, câmera, mouse, cinemática e interpolação.

Acesso aos objetivos

Busca no piso verifica se todos os resíduos e lixeiras são alcançáveis com as colisões.

> Protótipos STL são clonados; cenário e modelos são reutilizados no reinício. Testes de lógica não medem FPS nem substituem inspeção visual.

Demonstração: observar o conceito no jogo

01 **Explorar**
Andar, girar, arrastar e usar a roda: câmera e perspectiva.

02 **Chegar perto**
Encostar em um obstáculo; observar chão, materiais e sombras.

03 **Coletar**
Pressionar E e acompanhar agachamento, articulações e item nas mãos.

04 **Errar e acertar**
Mostrar −25, depois +100, abertura da tampa e arco do resíduo.

05 **Pausar e reiniciar**
M/Esc e R: congelamento da ação e restauração da partida.



Captura do jogo. Use o código como apoio para explicar o que está visível.

Um jogo pequeno conecta os quatro eixos

A coleta seletiva organiza uma experiência 3D com **malhas, transformações, câmera, luz, textura e animação.**

Lógica própria

Importação STL, malhas de revolução, normais, cinemática inversa, colisões e transições temporais.

Recurso da biblioteca

Primitivas, renderização, projeção e shader padrão; o código configura seus parâmetros.

Pontos a aprofundar

Grafo de cena explícito, coeficientes de material independentes e tratamento do tempo em baixo FPS.

> O domínio na defesa está em ligar o que aparece na tela ao cálculo que o produz — reconhecendo o alcance da implementação.

Apoios a seguir: mapa dos dez requisitos, matrizes, perguntas de defesa e fontes.

Os dez requisitos em uma página

REQUISITO DO ENUNCIADO	SLIDES	EVIDÊNCIA E RESSALVA
Coordenadas e transformações	05–06	Vetores e bases; matrizes apresentadas como equivalência teórica.
Modelagem 3D	07–08	Primitivas, STL e superfícies de revolução com normais.
Modelagem hierárquica	09	Encadeamento manual; sem grafo genérico de nós.
Câmera e projeções	10	Câmera configurada; projeção e divisão por w no renderizador.
Navegação	11	Teclado, mouse, proximidade e colisões aproximadas.
Iluminação	12	Quatro luzes direcionais e ambiente; shader padrão.
Sombreamento	13	Normais por face/vértice; comparação teórica Gouraud/Phong.
Materiais	14	Cor, brilho e opacidade; coeficientes separados não expostos.
Texturas	15	UV explícito nos caminhos e imagens em primitivas.
Animações	17–19	Δt e suavização; corte em 0,05 s limita o baixo FPS.

Matrizes: a forma explícita das operações

Vetores coluna; rotação positiva em Y pela regra da mão direita. c = cos θ; s = sin θ.

Translação T

$$\begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Rotação R_y

$$\begin{pmatrix} c & 0 & s & 0 \\ 0 & 1 & 0 & 0 \\ -s & 0 & c & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Escala S

$$\begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$M = T \cdot R_y \cdot S \quad | \quad p' = M \cdot (x, y, z, 1)^T$$

> Em uma articulação: T(pivô) · R · T(−pivô). No projeto, bases e pivôs codificam essas transformações sem montar matrizes 4 × 4.

Perguntas que conectam teoria e código

Se trocar rotação e translação?

O objeto pode girar em torno da origem do mundo; posição final e pivô percebido mudam.

Por que o distante fica menor?

A projeção usa a profundidade na coordenada w; a divisão por w reduz a extensão na imagem.

A animação independe totalmente do FPS?

Usa Δt; porém, o corte em 0,05 s descarta tempo quando o intervalo real passa desse limite.

Uma superfície suave tem mais polígonos?

Não necessariamente. Interpolar normais muda a iluminação sem alterar a silhueta da malha.

Clonar os STL garante desempenho?

Evita reconstruir a malha, mas FPS depende de objetos, triângulos, navegador e hardware. Não foi medido aqui.

Onde está o grafo de cena?

A dependência entre peças está nos cálculos da pose. Falta uma estrutura explícita de nós com herança automática.

Fontes e caminhos para conferir

Enunciado e implementação

Projeto Final — Computação Gráfica, seções 3 e 5.
`main.py`, módulos auxiliares, `tests/` e `README.md`.

Documentação oficial do VPython

Câmera · Luzes · Malhas, normais e UV · Texturas.

Projeção e shader

Khronos — OpenGL 4.6 Core, §13.8: divisão de perspectiva.
Shader distribuído com VPython 7.6.5: `vpython_libraries/glow.min.js` (lightAt e calc_color).

Modelos e evidências

Kenney Nature Kit 2.1 — CC0, conforme `kenney_nature-kit/License.txt`.
Captura e verificações desta revisão: `artifacts/apresentacao/`.

> Notas de fala: tecla N · Índice: tecla I · Tela cheia: F · Imprimir ou salvar PDF: P